

# Guide Finalisation Projet

## Étapes de la finalisation du projet :

- 1 - Finalisation autonome du service (deconnecté des autres services) **\*1**
- 2 - Finalisation autonome du service conteneurisé (deconnecté des autres services)
- 3 - Etablissement d'un docker compose global pour creer un environnement de tests local des differents services entre eux. (volumes globale...)
- 4 - Deployment Kubernetes (healthcheck, gracefull shutdown, persistant volumes, secrets, pods, cluster, replicas, gestions ressource CPU...) pour resilience, loadbalancing et scaling

---

## **\*1 - Fonctionnalités / Conditions a remplir pour chaques services**

### **Fonctionnement du service**

- **Scalabilité** : Le projet doit savoir gérer la charge, les services doivent donc pouvoir paralléliser les requêtes pour ne pas créer des files d'attentes interminable sous charge.
- **Pertinence des retours IA et Scan** : On doit pouvoir prouver la légitimité des retours de chaque services pour être certain que les retours ne disent pas n'importe quoi. (Arbitres, Tests avec des images plus ou moins connues / versions plus ou moins anciennes...)
- **Tests** : Pour ne pas passer a coté de bugs / comportements imprevis, il est nécessaire de tester son service avec une grande diversité d'éléments en entrée : Différentes Images, repository (ghcr.io ,registry-1.docker.io, quay.io...), taille d'image (50Mo, 2Go, 10Go...) client (podman / docker), payload (injections..), images corrompues, Simulation sous charge (grand nombre de requetes) ...
- **Sécurité** : Validation Inputs, Rate limiting, Validation cryptographique des digests, Taille limite des blobs...

- **CI/CD** : Workflow → Lint, Tests, Audit sécurité (RustSec), build, conteneurisation Docker (Dockerfile), scan Trivy, push

- **Explications Service / guide** : README.md pour chaque branches (même style que celui du proxy / scanner compliance)

## Prerequis pour Kubernetes

- ☐ Les healthchecks répondent correctement (exposer route /health )
- ☐ Les graceful shutdowns sont implémentés (eteindre proprement service après un signal SIGTERM)
- ☐ Logs envoyés sur la sortie standard
- ☐ Zéro config hardcodée dans le code, tout doit passer dans les .env + documentation pour chaque .env
- ☐ Les images sont toutes publiées sur ghcr.io avec des tags versionnés (CI/CD)
- ☐ Tests de bout en bout : docker pull fonctionne à travers le stack complet
- ☐ Tous les services passent par le docker-compose global (pas de volumes montés manuellement)

---

## Deadlines :

### 7 mai finalisation services

→ Tests de bout en bout : docker pull fonctionne à travers le stack complet

**Etapes suivantes** : docker-compose puis Kubernetes

---

## Amélioration ? :

- Communication sécurisé TLS entre les services
- Envoie d'un lien S3 (bucket cloud) au lieu d'envoyer tout les fichiers (lourd) entre services
- Service Surveillance continue, cycle vie images (SBOM + CVE = detection → relance scan)